CMPSC 448 Midterm Project

Meghana Vinjamury

1. Problem and dataset

The goal is to predict which LLM generated a response. I implemented a CNN and an LSTM to answer RQ1 using response text, then compared prompt-only, response-only, and combined inputs for RQ2.

The dataset uses real, publicly released AlpacaEval responses [1]. The three source generators are gpt4_0613 (GPT-4), claude-2 (Claude 2), and llama-2-7b-chat-hf (Llama 2 7B Chat). Each has 805 responses, giving 2,415 examples for 805 shared prompts. prepare_data.py downloads the model_outputs.json files from the pinned repository revision.

I normalized prompt whitespace, checked generator labels, removed empty records if present, and kept only prompts answered by all three models. No records in these source files were empty. The CSV contains LLM_name, LLM_Input, LLM_output, and split. It does not contain private chats. The dataset card lists CC BY-NC 4.0; this is a noncommercial course project, with attribution retained [2].

Prompts are shuffled with seed 42 and split approximately 70/15/15. All three responses to a prompt stay in one split. This avoids training and testing on responses to the same question. Classes are balanced in every split.

Split	Prompts	Responses	Responses per family
Training	563	1689	563
Validation	120	360	120
Test	122	366	122

2. Models and training

Text is lowercased and split into word and punctuation tokens. Literal model/provider names are replaced with a common marker to reduce obvious identification clues. Each condition uses a vocabulary of up to 12,000 tokens learned from its training text only; unseen tokens use an unknown-token ID. Prompts are limited to 96 tokens and responses to 192. Combined input uses prompt + separator + response, retaining the same response budget.

CNN: a trainable 64-dimensional embedding, one 1D convolution with 64 filters and a window of 3 tokens, ReLU, max pooling, dropout 0.3, and a three-class linear layer. LSTM: a trainable 64-dimensional embedding, one forward LSTM layer with 64 hidden units, mean pooling, dropout 0.3, and a three-class linear layer. Both exclude padding from pooling.

All six conditions train from scratch on CPU with seed 42. I used cross-entropy loss, Adam with learning rate 0.002 and weight decay 0.0001, batches of 64, and gradient clipping at norm 1. Training runs for at most 12 epochs, stops after 3 epochs without validation-loss improvement, and restores the lowest-validation-loss weights before evaluating the test set. Test scores are not used to choose weights.

3. Results and required comparisons

Model	Classifier input	Accuracy (%)	Macro-F1 (%)
CNN	Prompt only	33.33	31.59
CNN	Response only	85.79	85.86
CNN	Prompt + response	90.44	90.50
LSTM	Prompt only	33.33	32.76
LSTM	Response only	84.43	84.46
LSTM	Prompt + response	81.15	81.20

Each score comes from one seed on the same 366 test responses. Accuracy is the fraction of correct family labels. Macro-F1 averages the three class F1 scores equally, so performance on each family matters.

RQ1: identifying the model from its response

The response-only CNN achieved 85.79% accuracy and the LSTM achieved 84.43%. Both were above the balanced chance baseline of 33.33%. This supports identifying the three selected models from their responses in this dataset. The scores do not establish identification of arbitrary LLMs or newer versions.

RQ2: does including the prompt help?

Prompt-only accuracy was exactly 33.33% for both models. Each test prompt occurs with all three labels. A deterministic classifier makes the same prediction for all three identical inputs, so exactly one prediction is correct. Prompt-only macro-F1 can differ from 33.33% because predictions do not have to be equally distributed among the classes.

Adding the prompt changed CNN accuracy by +4.64 percentage points and LSTM accuracy by -3.28 points. The CNN benefited in this run, while the LSTM did worse. One possible explanation is that extra prompt tokens dilute response information in the LSTM mean-pooled representation. This experiment does not prove that explanation. High prompt-only accuracy in a less balanced dataset could instead reflect topic differences between models or data leakage.

4. Limitations and lessons

Only one version per family, one split, and one training seed were tested. Model versions, provider templates, and decoding settings can contribute to the differences. Longer responses are truncated, and near-duplicate prompts were not fully checked. Although literal names are masked, other identity clues may remain. There were two repeated response strings, both within training; no exact response strings overlapped different splits. The classifier always chooses one of the three labels and cannot recognize an unknown model.

The main lesson is that a fair comparison requires controlling the dataset and input preparation. Keeping each prompt in one split prevents leakage. Learning the vocabulary only from training data prevents using test information. The combined-input results also show that adding text does not automatically improve a classifier. Validation is needed to select the checkpoint, and the separate test set is needed for the final comparison.

5. Sources and assistance

[1] Li et al. (2023). AlpacaEval: An Automatic Evaluator of Instruction-following Models. https://github.com/tatsu-lab/alpaca_eval

[2] Dataset card and license: https://huggingface.co/datasets/tatsu-lab/alpaca_eval (CC BY-NC 4.0).